

CONTENTS

1. Process Management-I
(Introduction, Process, Threads & CPU Scheduling) 561
2. Process Management-II
(IPC, Synchronization and Concurrency) 580
3. Deadlock 598
4. Memory Management and Virtual Memory 606
5. File System and Device Management 626
6. Miscellaneous 635

Operating System

Syllabus

System calls, Processes, Threads, Inter-process communication, Concurrency, Synchronization, Deadlock, CPU and I/O scheduling, Memory management and virtual memory, File systems, I/O systems.

Analysis of Previous GATE Papers

Exam Year	1 Mark Ques.	2 Marks Ques.	Total Marks
1997	8	2	12
1998	8	2	12
1999	3	3	9
2000	1	2	5
2001	10	3	16
2002	2	4	10
2003	2	5	12
2004	3	4	11
2005	—	2	4
2006	1	8	17
2007	2	6	14
2008	2	5	12
2009	2	5	12
2010	3	2	7
2011	4	2	8
2012	1	4	9
2013	2	4	10
2014 Set-1	2	3	8
2014 Set-2	1	3	7
2014 Set-3	1	3	7

Exam Year	1 Mark Ques.	2 Marks Ques.	Total Marks
2015 Set-1	2	4	10
2015 Set-2	2	3	8
2015 Set-3	2	2	6
2016 Set-1	1	4	9
2016 Set-2	1	3	7
2017 Set-1	2	2	6
2017 Set-2	2	2	6
2018	2	3	8
2019	2	4	10
2020	2	4	10
2021 Set-1	4	1	6
2021 Set-2	1	3	7
2022	2	4	10
2023	3	3	9
2024 Set-1	2	4	10
2024 Set-2	2	4	10
2025 Set-1	2	3	8
2025 Set-2	1	3	7
2026 Set-1	2	3	8
2026 Set-2	1	4	9

1

Process Management-I

(Introduction, Process, Threads & CPU Scheduling)

1.1 Which of the following is an example of a spooled device?

- (a) The terminal used to enter the input data for the C program being executed
- (b) An output device used to print the output of a number of jobs.
- (c) The secondary memory device in a virtual storage system
- (d) The swapping area on a disk used by the swapper.

[1998 : 1 M]

1.2 Consider n processes sharing the CPU in a Round-Robin fashion. Assuming that each process switch takes s seconds, what must be the quantum size q such that the overhead resulting from process switching is minimized but at the same time each process is guaranteed to get its turn at the CPU at least every t seconds?

- (a) $q \leq \frac{t - ns}{n - 1}$
- (b) $q \geq \frac{t - ns}{n - 1}$
- (c) $q \leq \frac{t - ns}{n + 1}$
- (d) $q \geq \frac{t - ns}{n + 1}$

[1998 : 1 M]

1.3 Four jobs are waiting to be run. Their expected run times are 6, 3, 5 and x in what order should they be run to minimize the average response time?

[1998 : 2 M]

1.4 System calls are usually invoked by using

- (a) a software interrupt
- (b) polling
- (c) an indirect jump
- (d) a privileged instruction

[1999 : 1 M]

1.5 A multi-user, multi-processing operating system cannot be implemented on hardware that does not support

- (a) Address translation
- (b) DMA for disk transfer
- (c) At least two modes of CPU execution (privileged and non-privileged)
- (d) Demand paging

[1999 : 2 M]

1.6 Which of the following actions is/are typically not performed by the operating system when switching context from process A to process B?

- (a) Saving current register values and restoring saved register values for process B.
- (b) Changing address translation tables.

(c) Swapping out the memory image of process A to the disk.

(d) Invalidating the translation look-aside buffer.

[1999 : 2 M]

1.7 A processor needs software interrupt to

- (a) Test the interrupt system of the processor
- (b) Implement co-routines
- (c) Obtain system services which need execution of privileged instructions
- (d) Return from subroutine

[2001 : 1 M]

1.8 A CPU has two modes-privileged and non-privileged. In order to change the mode from privileged to non-privileged

- (a) a hardware interrupt is needed
- (b) a software interrupt is needed
- (c) a privileged instruction (which does not generate an interrupt) is needed
- (d) a non-privileged instruction (which does not generate an interrupt) is needed

[2001 : 1 M]

1.9 Consider a set of n tasks with known runtimes $r_1, r_2, \dots, r_n$ to be run on a uniprocessor machine. Which of the following processor scheduling algorithms will result in the maximum throughput?

- (a) Round-Robin
- (b) Shortest-Job-First
- (c) Highest-Response-Ratio-Next
- (d) First-Come-First-Served

[2001 : 1 M]

1.10 Which of the following scheduling algorithms is non-preemptive?

- (a) Round Robin
- (b) First-in-First-out
- (c) Multilevel Queue Scheduling
- (d) Multilevel Queue Scheduling with Feedback

[2001 : 1 M]

1.11 Which of the following does not interrupt a running process?

- (a) A device
- (b) Timer
- (c) Scheduler process
- (d) Power failure

[2001 : 2 M]

- 1.12** Which combination of the following features will suffice to characterize an OS as a multi-programmed OS? (i) more than one program may be loaded into main memory at the same time for execution. (ii) If a program waits for certain events such as I/O, another program is immediately scheduled for execution. (iii) If the execution of a program terminates, another program is immediately scheduled for execution
- (a) (i) (b) (i) and (ii)
(c) (i) and (iii) (d) (i), (ii) and (iii)
- [2002 : 2 M]

- 1.13** Draw the process state transition diagram of an OS in which (i) each process is in one of the five states: created, ready, running, blocked (i.e. sleep or wait), or terminated, and (ii) only non-preemptive scheduling is used by the OS. Label the transitions appropriately.
- [2002 : 2 M]

- 1.14** A uni-processor computer system only has two processes, both of which alternate 10 ms CPU bursts with 90 ms I/O bursts. Both the processes were created at nearly the same time. The I/O of both processes can proceed in parallel. Which of the following scheduling strategies will result in the least CPU utilization (over a long period of time) for this system?
- (a) First come first served scheduling
(b) Shortest remaining time first scheduling
(c) Static priority scheduling with different priorities for the two processes
(d) Round robin scheduling with a time quantum of 5 ms.
- [2003 : 2 M]

- 1.15** Consider the following statements with respect to user-level threads and kernel-supported threads
- (i) Context switch is faster with Kernel-supported threads
(ii) For user-level threads, a system call can block the entire process
(iii) Kernel-supported threads can be scheduled independently
(iv) User-level threads are transparent to the Kernel
- Which of the above statements are true?
- (a) (ii), (iii) and (iv) only
(b) (ii) and (iii) only (c) (i), and (iii) only
(d) (i) and (ii) only
- [2004 : 1 M]

- 1.16** Which one of the following is NOT shared by the threads of the same process?
- (a) Stack (b) Address Space

- (c) File Descriptor Table
(d) Message Queue

[2004 : 1 M]

- 1.17** A process executes the following segment of code:
- ```
for(i = 1; i <= n; i++)
 fork();
```

The number of new processes created is

- (a) n (b)  $((n(n+1))/2)$   
(c)  $2^n - 1$  (d)  $3^n - 1$
- [2004 : 2 M]

- 1.18** Consider the following set of processes, with the arrival times and the CPU-burst times given in milliseconds

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| P1      | 0            | 5          |
| P2      | 1            | 3          |
| P3      | 2            | 3          |
| P4      | 4            | 1          |

What is the average turnaround time for these processes with the preemptive Shortest Remaining Processing Time first (SRPT) algorithm?

- (a) 5.50 (b) 5.75  
(c) 6.00 (d) 6.25
- [2004 : 2 M]

- 1.19** We wish to schedule three processes P1, P2 and P3 on a uniprocessor system. The priorities, CPU time requirements and arrival times of the processes are as shown below:

| Process | Priority     | CPU time required | Arrival time (hh:mm:ss) |
|---------|--------------|-------------------|-------------------------|
| P1      | 10 (highest) | 20 sec            | 00:00:05                |
| P2      | 9            | 10 sec            | 00:00:03                |
| P3      | 8 (lowest)   | 15 sec            | 00:00:00                |

We have a choice of preemptive or non-preemptive scheduling. In preemptive scheduling, a late-arriving higher priority process can preempt a currently running process with lower priority. In non-preemptive scheduling, a late-arriving higher priority process must wait for the currently executing process to complete before it can be scheduled on the processor. What are the turnaround times (time from arrival till completion) of P2 using preemptive and non-preemptive scheduling respectively?

- (a) 30 sec, 30 sec (b) 30 sec, 10 sec  
(c) 42 sec, 42 sec (d) 30 sec, 42 sec
- [2005 : 2 M]

1.20 Two shared resources  $R_1$  and  $R_2$  are used by processes  $P_1$  and  $P_2$ . Each process has a certain priority for accessing each resource. Let  $T_{ij}$  denote the priority of  $P_i$  for accessing  $R_j$ . A process  $P_i$  can snatch a resource  $R_k$  from process  $P_j$  if  $T_{ik}$  is greater than  $T_{jk}$ .

Given the following:

- I.  $T_{11} > T_{21}$       II.  $T_{12} > T_{22}$   
 III.  $T_{11} < T_{21}$       IV.  $T_{12} < T_{22}$

Which of the following conditions ensures that  $P_1$  and  $P_2$  can never deadlock?

- (a) (I) and (IV)  
 (b) (II) and (III)  
 (c) (I) and (II)  
 (d) None of these

[2005 : 2 M]

1.21 Consider the following code fragment:

```
if (fork () == 0)
```

```
{a = a + 5; printf("%d, %d\n", a, &a);}
```

```
else{a = a - 5; printf("%d, %d\n", a, &a);}
```

Let  $u, v$  be the values printed by the parent process, and  $x, y$  be the values printed by the child process.

Which one of the following is TRUE?

- (a)  $u = x + 10$  and  $v = y$   
 (b)  $u = x + 10$  and  $v \neq y$   
 (c)  $u + 10 = x$  and  $v = y$   
 (d)  $u + 10 = x$  and  $v \neq y$

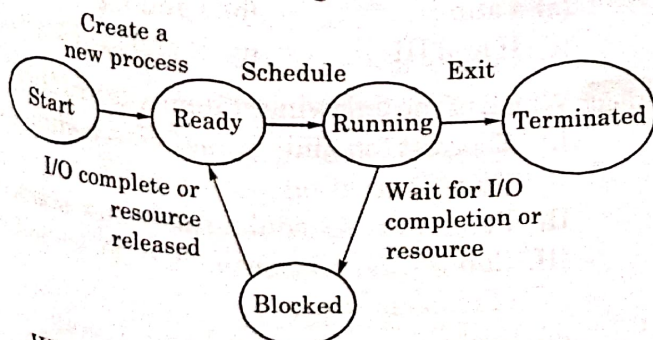
[2005 : 2 M]

1.22 Consider three CPU-intensive processes, which require 10, 20 and 30 time units and arrive at times 0, 2, and 6, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end

- (a) 1      (b) 2  
 (c) 3      (d) 4

[2006 : 1 M]

1.23 The process state transition diagram of an operating system is as given below.



Which of the following must be FALSE about the above operating system?

- (a) It is a multiprogrammed operating system  
 (b) It uses preemptive scheduling  
 (c) It uses non-preemptive scheduling  
 (d) It is a multi-user operating system

[2006 : 1 M]

1.24 The arrival time, priority, and durations of the CPU and I/O bursts for each of three processes  $P_1, P_2$  and  $P_3$  are given in the table below. Each process has a CPU burst followed by an I/O burst followed by another CPU burst. Assume that each process has its own I/O resource.

| Process | Arrival time | Priority       | Burst durations<br>CPU, I/O CPU |
|---------|--------------|----------------|---------------------------------|
| $P_1$   | 0            | 2              | 1, 5, 3                         |
| $P_2$   | 2            | 3<br>(lowest)  | 3, 3, 1                         |
| $P_3$   | 3            | 1<br>(highest) | 2, 3, 1                         |

The multi-programmed operating system uses preemptive priority scheduling. What are the finish times of the processes  $P_1, P_2$  and  $P_3$ ?

- (a) 11, 15, 9      (b) 10, 15, 9  
 (c) 11, 16, 10      (d) 12, 17, 11

[2006 : 2 M]

1.25 Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process get blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

- (a) 0%      (b) 10.6%  
 (c) 30.0%      (d) 89.4%

[2006 : 2 M]

1.26 Consider three processes (process id 0, 1, 2, respectively) with compute time bursts 2, 4, and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turn around time is

- (a) 13 units      (b) 14 units  
 (c) 15 units      (d) 16 units

[2006 : 2 M]

- 1.27** List-I contains some CPU scheduling algorithms and List-II contains some applications. Match entries in List-I entries in List-II

**List-I**

- A. Gang Scheduling
- B. Rate Monotonic Scheduling
- C. Fair Share Scheduling

**List-II**

- 1. Guaranteed Scheduling
- 2. Real-time Scheduling
- 3. Thread Scheduling

Code:

- |     | A | B | C |
|-----|---|---|---|
| (a) | 3 | 2 | 1 |
| (b) | 1 | 2 | 3 |
| (c) | 2 | 3 | 1 |
| (d) | 1 | 3 | 2 |

[2007 : 1 M]

- 1.28** Consider the following statements about user level threads and kernel level threads. Which one of the following statements is FALSE?

- (a) Context switch time is longer for Kernel level threads than for user level threads
- (b) User level threads do not need any hardware support
- (c) Related Kernel level threads can be scheduled on different processors in a multiprocessor system
- (d) Blocking one Kernel level thread blocks all related threads

[2007 : 1 M]

- 1.29** An operating system uses Shortest Remaining Time first (SRT) process scheduling algorithm. Consider the arrival times and execution times for the following processes:

| Process | Execution time | Arrival time |
|---------|----------------|--------------|
| P1      | 20             | 0            |
| P2      | 25             | 15           |
| P3      | 10             | 30           |
| P4      | 15             | 45           |

What is the total waiting time for process P2?

- (a) 5
- (b) 15
- (c) 40
- (d) 55

[2007 : 2 M]

- 1.30** If the time-slice used in the round-robin scheduling policy is more than the maximum time required to execute any process, then the policy will

- (a) degenerate to shortest job first
- (b) degenerate to priority scheduling
- (c) degenerate to first come first serve
- (d) None of the above

[2008 : 2 M]

- 1.31** Which of the following statements about synchronous and asynchronous I/O is NOT true?
- (a) An ISR is invoked on completion of I/O in synchronous I/O but not in asynchronous I/O.
  - (b) In both synchronous and asynchronous I/O an ISR (Interrupt Service Routine) is invoked after completion of the I/O.
  - (c) A process making a synchronous I/O call waits until I/O is complete, but a process making an asynchronous I/O call does not wait for completion of the I/O.
  - (d) In the case of synchronous I/O, the process waiting for the completion of I/O is woken up by the ISR that is invoked after the completion of I/O.

[2008 : 2 M]

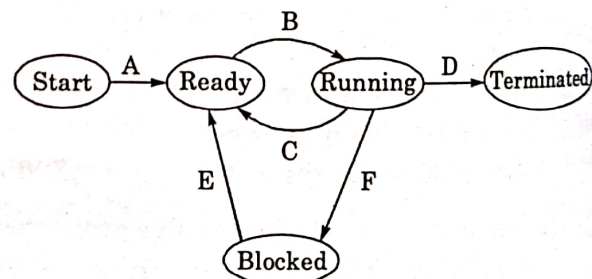
- 1.32** A process executes the following code for  $(i = 0; i < n; i++)$  fork ();

The total number of child processes created is

- (a)  $n$
- (b)  $2^n - 1$
- (c)  $2^n$
- (d)  $2^{n+1} - 1$

[2008 : 2 M]

- 1.33** In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state:



Now consider the following statements:

- I. If a process makes a transition D, it would result in another process making transition A immediately.
- II. A process  $P_2$  in blocked state can make transition E while another process  $P_1$  is in running state.
- III. The OS uses preemptive scheduling.
- IV. The OS uses non-preemptive scheduling.

Which of the above statements are TRUE?

- (a) I and II
- (b) I and III
- (c) II and III
- (d) II and IV

[2009 : 2 M]

- 1.34** Which of the following statements are true?
- I. Shortest remaining time first scheduling may cause starvation.
  - II. Preemptive scheduling may cause starvation.
  - III. Round-Robin is better than FCFS in terms of response time.

- (a) I only
- (b) I and III only
- (c) II and III only
- (d) I, II and III

[2010 : 1 M]

**1.35** Let the time taken to switch between user and Kernel modes of execution be  $t_1$  while the time taken to switch between two processes be  $t_2$ . Which of the following is TRUE?

- (a)  $t_1 > t_2$  (b)  $t_1 = t_2$   
 (c)  $t_1 < t_2$   
 (d) Nothing can be said about the relation between  $t_1$  and  $t_2$  [2011 : 1 M]

**1.36** A computer handles several interrupt sources of which the following are relevant for this question.

- Interrupt from CPU temperature sensor (raises interrupt if CPU temperature is too high)
- Interrupt from Mouse (raises interrupt if the mouse is moved or a button is pressed)
- Interrupt from Keyboard (raises interrupt when a key is pressed or released)
- Interrupt from Hard Disk (raises interrupt when a disk read is completed)

Which one of these will be handled at the HIGHEST priority?

- (a) Interrupt from Hard Disk  
 (b) Interrupt from Mouse  
 (c) Interrupt from Keyboard  
 (d) Interrupt from CPU temperature sensor [2011 : 1 M]

**1.37** A thread is usually defined as a "light weight process" because an Operating System (OS) maintains smaller data structures for a thread than for a process. In relation to this, which of the following is TRUE?

- (a) On per-thread basis, the OS maintains only CPU register state  
 (b) The OS does not maintain a separate stack for each thread  
 (c) On per-thread basis, the OS does not maintain virtual memory state  
 (d) On per-thread basis, the OS maintains only scheduling and accounting information [2011 : 1 M]

**1.38** Consider the following table of arrival time and burst time for three processes P0, P1 and P2.

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| P0      | 0 ms         | 9 ms       |
| P1      | 1 ms         | 4 ms       |
| P2      | 2 ms         | 9 ms       |

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

- (a) 5.0 ms (b) 4.33 ms  
 (c) 6.33 ms (d) 7.33 ms [2011 : 2 M]

**1.39** A process executes the code:

```
fork();
fork();
fork();
```

The total number of child processes created is

- (a) 3 (b) 4  
 (c) 7 (d) 8 [2012 : 1 M]

**1.40** Consider the 3 processes, P1, P2 and P3 shown in the table:

| Process Name | Arrival Time | Time Unit Required |
|--------------|--------------|--------------------|
| P1           | 0            | 5                  |
| P2           | 1            | 7                  |
| P3           | 3            | 4                  |

The completion order of the 3 processes under the policies FCFS and RR2 (Round Robin scheduling with CPU quantum of 2 time units) are:

- (a) FCFS: P1, P2, P3 RR2: P1, P2, P3  
 (b) FCFS: P1, P3, P2 RR2: P1, P3, P2  
 (c) FCFS: P1, P2, P3 RR2: P1, P3, P2  
 (d) FCFS: P1, P3, P2 RR2: P1, P2, P3 [2012 : 2 M]

**1.41** A scheduling algorithm assigns priority proportional to the waiting time of a process. Every process starts with priority zero (the lowest priority). The schedule reevaluates the process priorities every T time units and decides the next process to schedule. Which one of the following is TRUE if the processes have no I/O operations and all arrive at time zero?

- (a) This algorithm is equivalent to the first come first serve algorithm  
 (b) This algorithm is equivalent to the round-round algorithm  
 (c) This algorithm is equivalent to the shortest-job-first algorithm  
 (d) This algorithm is equivalent to the shortest-remaining-time-first algorithm [2013 : 1 M]

**1.42** Which one of the following is FALSE?

- (a) User level threads are not scheduled by the Kernel.  
 (b) When a user level thread is blocked, all other threads of its process are blocked.  
 (c) Context switching between user level threads is faster than context switching between Kernel level threads.  
 (d) Kernel level threads cannot share the code segment. [2014 (Set-1) : 1 M]

- 1.43** Consider the following set of processes that need to be scheduled on a single CPU. All the times are given in milliseconds.

| Process Name | Arrival Time | Execution Time |
|--------------|--------------|----------------|
| A            | 0            | 6              |
| B            | 3            | 2              |
| C            | 5            | 4              |
| D            | 7            | 6              |
| E            | 10           | 3              |

Using the shortest remaining time first scheduling algorithm, the average process turnaround time (in msec) is \_\_\_\_\_.

[2014 (Set-1) : 2 M]

- 1.44** Three processes A, B and C each execute a loop of 100 iterations. In each iteration of the loop, a process performs a single computation that requires  $t_c$  CPU milliseconds and then initiates a single I/O operation that lasts for  $t_{io}$  milliseconds. It is assumed that the computer where the processes execute has sufficient number of I/O devices and the OS of the computer assigns different I/O devices to each process. Also, the scheduling overhead of the OS is negligible. The processes have the following characteristics:

| Process id | $t_c$  | $t_{io}$ |
|------------|--------|----------|
| A          | 100 ms | 500 ms   |
| B          | 350 ms | 500 ms   |
| C          | 200 ms | 500 ms   |

The processes A, B, and C are started at times 0, 5 and 10 milliseconds respectively, in a pure time sharing system (round robin scheduling) that uses a time slice of 50 milliseconds. The time in milliseconds at which process C would complete its first I/O operation is \_\_\_\_\_.

[2014 (Set-2) : 2 M]

- 1.45** An operating system uses shortest remaining time first scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in milliseconds):

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 12         |
| P2      | 2            | 4          |
| P3      | 3            | 6          |
| P4      | 8            | 5          |

The average waiting time (in milliseconds) of the processes is \_\_\_\_\_.

[2014 (Set-3) : 2 M]

- 1.46** Consider a uniprocessor system executing three tasks  $T_1$ ,  $T_2$  and  $T_3$ , each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. The priority of each task is the inverse of its period, and the available tasks are scheduled in order of priority, with the highest priority task scheduled first. Each instance of  $T_1$ ,  $T_2$  and  $T_3$  requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1<sup>st</sup> millisecond and task preemptions are allowed, the first instance of  $T_3$  completes its execution at the end of \_\_\_\_\_ milliseconds.

[2015 (Set-1) : 2 M]

- 1.47** The maximum number of processes that can be in Ready state for a computer system with  $n$  CPUs is
- $n$
  - $n^2$
  - $2^n$
  - Independent of  $n$

[2015 (Set-3) : 1 M]

- 1.48** For the processes listed in the following table, which of the following scheduling schemes will give the lowest average turnaround time?

| Process | Arrival Time | Processing Time |
|---------|--------------|-----------------|
| A       | 0            | 3               |
| B       | 1            | 6               |
| C       | 4            | 4               |
| D       | 6            | 2               |

- First Come First Serve
- Non-preemptive Shortest Job First
- Shortest Remaining Time
- Round-Robin with Quantum value two

[2015 (Set-3) : 2 M]

- 1.49** Consider an arbitrary set of CPU-bound processes with unequal CPU burst lengths submitted at the same time to a computer system. Which one of the following process scheduling algorithms would minimize the average waiting time in the ready queue?

- Shortest remaining time first
- Round-robin with time quantum less than the shortest CPU burst
- Uniform random
- Highest priority first with priority proportional to CPU burst length

[2016 (Set-1) : 1 M]

- 1.50** Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. The scheduling algorithm used is preemptive shortest remaining-time first.

| Process        | Arrival Time | Burst Time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0            | 10         |
| P <sub>2</sub> | 3            | 6          |
| P <sub>3</sub> | 7            | 1          |
| P <sub>4</sub> | 8            | 3          |

The average turn around time of these processes is \_\_\_\_\_ milliseconds.

[2016 (Set-2) : 2 M]

- 1.51** Threads of a process share

- global variables but not heap
- heap but not global variables
- neither global variables nor heap
- both heap and global variables

[2017 (Set-1) : 1 M]

- 1.52** Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) as given below:

| Process        | Arrival time | Burst time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0            | 7          |
| P <sub>2</sub> | 3            | 3          |
| P <sub>3</sub> | 5            | 5          |
| P <sub>4</sub> | 6            | 2          |

If the pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then the average waiting time across all processes is \_\_\_\_\_ milliseconds.

[2017 (Set-1) : 1 M]

- 1.53** Which of the following is/are shared by all the threads in a process?

- Program counter
- Stack
- Address space
- Registers

- I and II only
- III only
- IV only
- III and IV only

[2017 (Set-2) : 1 M]

- 1.54** Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority (0 is the highest priority) shown below. None of the processes have I/O burst time.

| Process        | Arrival Time | Burst Time | Priority |
|----------------|--------------|------------|----------|
| P <sub>1</sub> | 0            | 11         | 2        |
| P <sub>2</sub> | 5            | 28         | 0        |
| P <sub>3</sub> | 12           | 2          | 3        |
| P <sub>4</sub> | 2            | 10         | 1        |
| P <sub>5</sub> | 9            | 16         | 4        |

The average waiting time (in milliseconds) of all the processes using preemptive priority scheduling algorithm is \_\_\_\_\_.

[2017 (Set-2) : 2 M]

- 1.55** Consider the following C program is executed on a Unix/Linux system:

```
#include<unistd.h>
int main()
{
 int i;
 for (i = 0; i < 10; i++)
 if (i % 2 == 0) fork();
 return 0;
}
```

The total number of child processes created is \_\_\_\_\_.

[2019 : 1 M]

- 1.56** Consider the following four processes with arrival times (in milliseconds) and their length of CPU burst (in milliseconds) as shown below:

| Process      | P <sub>1</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>4</sub> |
|--------------|----------------|----------------|----------------|----------------|
| Arrival time | 0              | 1              | 3              | 4              |
| CPU time     | 3              | 1              | 3              | Z              |

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is \_\_\_\_\_.

[2019 : 2 M]

- 1.57** Consider the following statements about process state transitions for a system using preemptive scheduling.

- A running process can move to ready state.
- A ready process can move to running state.
- A blocked process can move to running state.
- A blocked process can move to ready state.

Which of the above statements are TRUE?

- I, II, III and IV
- II and III only
- I, II and III only
- I, II and IV only

[2020 : 1 M]

- 1.58** Consider the following set of processes, assumed to have arrived at time 0. Consider the CPU scheduling algorithms Shortest Job First (SJF) and Round Robin (RR). For RR, assume that the processes are scheduled in the order  $P_1, P_2, P_3, P_4$ .

| Processes          | $P_1$ | $P_2$ | $P_3$ | $P_4$ |
|--------------------|-------|-------|-------|-------|
| Burst time (in ms) | 8     | 7     | 2     | 4     |

If the time quantum for RR is 4 ms, then the absolute value of the difference between the average turnaround times (in ms) of SJF and RR (round off to 2 decimal places) is \_\_\_\_\_.

[2020 : 2 M]

- 1.59** Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the length of the CPU bursts, the minimum achievable average waiting time for these three processes in a non-preemptive scheduler (round to nearest integer) is \_\_\_\_\_ milliseconds.

[2021 (Set-1) : 1 M]

- 1.60** Which of the following standard C library functions will always invoke a system call when executed from a single-threaded process in a UNIX/Linux operating system?

- (a) exit                      (b) strlen  
(c) sleep                    (d) malloc

[2021 (Set-1) : 1 M]

- 1.61** Which of the following statement(s) is/are correct in the context of CPU scheduling?

- (a) The goal is to only maximize CPU utilization and minimize throughput.  
(b) Turnaround time includes waiting time.  
(c) Round-robin policy can be used even when the CPU time required by each of the processes is not known apriori.  
(d) Implementing preemptive scheduling needs hardware support.

[2021 (Set-2) : 1 M]

- 1.62** Consider four processes P, Q, R, and S scheduled on a CPU as per round robin algorithm with a time quantum of 4 units. The processes arrive in the order P, Q, R, S, all at time  $t = 0$ . There is exactly one context switch from S to Q, exactly one context switch from R to Q, and exactly two context switches from Q to R. There is no context switch from S to P. Switching to a ready process after the termination of another process is also

considered a context switch. Which one of the following is NOT possible as CPU burst time (in time units) of these processes?

- (a)  $P = 4, Q = 10, R = 6, S = 2$   
(b)  $P = 2, Q = 9, R = 5, S = 1$   
(c)  $P = 4, Q = 12, R = 5, S = 4$   
(d)  $P = 3, Q = 7, R = 7, S = 3$

[2022 : 2 M]

- 1.63** Which one or more of the following need to be saved on a context switch from one thread (T1) of a process to another thread (T2) of the same process?

- (a) Page table base register  
(b) Stack pointer  
(c) Program counter  
(d) General purpose registers

[2023 : 1 M]

- 1.64** Which one or more of the following options guarantee that a computer system will transition from user mode to Kernel mode?

- (a) Function call              (b) Malloc call  
(c) Page fault                (d) System call

[2023 : 1 M]

- 1.65** Which one or more of the following CPU scheduling algorithms can potentially cause starvation?

- (a) First-in First-Out  
(b) Round Robin  
(c) Priority Scheduling  
(d) Shortest Job First

[2023 : 1 M]

- 1.66** Which of the following statements about threads is/are TRUE?

- (a) Threads can only be implemented in Kernel space  
(b) Each thread has its own file descriptor table for open files  
(c) Threads belonging to a process are by default not protected from each other  
(d) All the threads belonging to a process share a common stack

[2024 (Set-1) : 1 M]

- 1.67** Which of the following process state transitions is/are NOT possible?

- (a) Waiting to Running  
(b) Ready to Waiting  
(c) Running to Terminated  
(d) Running to Ready

[2024 (Set-1) : 1 M]

- 1.68 Consider the following two threads T1 and T2 that update two shared variables  $a$  and  $b$ . Assume that initially  $a = b = 1$ . Though context switching between threads can happen at any time, each statement of T1 or T2 is executed atomically without interruption.

| T1           | T2           |
|--------------|--------------|
| $a = a + 1;$ | $b = 2 * b;$ |
| $b = b + 1;$ | $a = 2 * a;$ |

Which one of the following options lists all the possible combinations of values of  $a$  and  $b$  after both T1 and T2 finish execution?

- (a)  $(a = 3, b = 4); (a = 4, b = 3); (a = 3, b = 3)$   
 (b)  $(a = 4, b = 4); (a = 4, b = 3); (a = 3, b = 4)$   
 (c)  $(a = 2, b = 2); (a = 2, b = 3); (a = 3, b = 4)$   
 (d)  $(a = 4, b = 4); (a = 3, b = 3); (a = 4, b = 3)$

[2024 (Set-1) : 2 M]

- 1.69 Consider a process P running on a CPU. Which one or more of the following events will always trigger a context switch by the OS that results in process P moving to a non-running state (e.g., ready, blocked)?

- (a) A timer interrupt is raised by the hardware.  
 (b) P tries to access a page that is in the swap space, triggering a page fault.  
 (c) An interrupt is raised by the disk to deliver data requested by some other process.  
 (d) P makes a blocking system call to read a block of data from the disk.

[2024 (Set-2) : 1 M]

- 1.70 Consider a multi-threaded program with two threads T1 and T2. The threads share two semaphores:  $s_1$  (initialized to 1) and  $s_2$  (initialized to 0). The threads also share a global variable  $x$  (initialized to 0). The threads execute the code shown below.

| // code of T1    | // code of T2    |
|------------------|------------------|
| wait ( $s_1$ );  | wait ( $s_1$ );  |
| $x = x + 1;$     | $x = x + 1;$     |
| print( $x$ );    | print( $x$ );    |
| wait( $s_2$ );   | signal( $s_2$ ); |
| signal( $s_1$ ); | signal( $s_1$ ); |

Which of the following outcomes is/are possible when threads T1 and T2 execute concurrently?

- (a) T2 runs first and prints 1, T1 runs next and prints 2  
 (b) T2 runs first and prints 1, T1 does not print anything (deadlock)

- (c) T1 runs first and prints 1, T2 runs next and prints 2  
 (d) T1 runs first and prints 1, T2 does not print anything (deadlock)

[2024 (Set-2) : 2 M]

- 1.71 Consider a single processor system with four processes A, B, C, and D, represented as given below, where for each process the first value is its arrival time, and the second value is its CPU burst time.

A(0, 10), B(2, 6), C(4, 3) and D(6, 7)

Which one of the following options gives the average waiting times when preemptive Shortest Remaining Time First (SRTF) and Non-Preemptive Shortest Job First (NP-SJF) CPU scheduling algorithms are applied to the processes?

- (a) SRTF = 6, NP-SJF = 7  
 (b) SRTF = 7, NP-SJF = 7.5  
 (c) SRTF = 7, NP-SJF = 8.5  
 (d) SRTF = 6, NP-SJF = 7.5

[2024 (Set-2) : 2 M]

- 1.72 Suppose in a multiprogramming environment, the following C program segment is executed. A process goes into I/O queue whenever an I/O related operation is performed. Assume that there will always be a context switch whenever a process requests for an I/O, and also whenever the process returns from an I/O. The number of times the process will enter the ready queue during its lifetime (not counting the time the process enters the ready queue when it is run initially) is \_\_\_\_\_. (Answer in integer)

```
int main()
{
 int x = 0, i = 0;
 scanf("%d", &x);
 for(i = 0; i < 20; i++)
 {
 x = x + 20;
 printf("%d\n", x);
 }
 return 0;
}
```

[2025 (Set-1) : 1 M]

- 1.73 A computer has two processors,  $M_1$  and  $M_2$ . Four processes  $P_1, P_2, P_3, P_4$  with CPU bursts of 20, 16, 25, and 10 milliseconds, respectively, arrive at the same time and these are the only processes

in the system. The scheduler uses non-preemptive priority scheduling, with priorities decided as follows:

- $M_1$  uses priority of execution for the processes as,  $P_1 > P_3 > P_2 > P_4$ , i.e.,  $P_1$  and  $P_4$  have highest and lowest priorities, respectively.
- $M_2$  uses priority of execution for the processes as,  $P_2 > P_3 > P_4 > P_1$ , i.e.,  $P_2$  and  $P_1$  have highest and lowest priorities, respectively.

A process  $P_i$  is scheduled to a processor  $M_k$ , if the processor is free and no other process  $P_j$  is waiting with higher priority. At any given point of time, a process can be allocated to any one of the free processors without violating the execution priority rules. Ignore the context switch time. What will be the average waiting time of the processes in milliseconds?

- 9.00
- 8.75
- 6.50
- 7.50

[2025 (Set-1) : 2 M]

**1.74** Processes  $P_1, P_2, P_3, P_4$  arrive in that order at times 0, 1, 2, and 8 milliseconds respectively, and have execution times of 10, 13, 6, and 9 milliseconds respectively. Shortest Remaining Time First (SRTF) algorithm is used as the CPU scheduling policy. Ignore context switching times. Which ONE of the following correctly gives the average turnaround time of the four processes in milliseconds?

- 22
- 15
- 37
- 19

[2025 (Set-2) : 1 M]

**1.75** Consider the following program snippet. Assume that the program compiles and runs successfully. Further, assume that the `fork()` system call is always successful in creating a process.

```
int main() {
 int i;
 for (i = 0; i < 3; i++){
 if (fork() == 0){
 continue;
 }
 break;
 }
 printf("Hello!");
 return 0;
}
```

The total number of times that the `printf` statement gets executed is \_\_\_\_\_. (answer in integer)

[2026 (Set-1) : 2 M]

**1.76** Consider a CPU that has to execute two types of processes. The first type, Actuators (A), requires a CPU burst of 6 seconds. The second type, Controllers (C), requires a CPU burst of 8 seconds. A new process of type A arrives at time  $t = 10, 20, 30, 40$ , and 50 (in seconds). Similarly, a new process of type C arrives at time  $t = 11, 22, 33, 44$ , and 55 (in seconds). The CPU scheduling policy is First Come First Serve (FCFS). The first process of type A starts running at  $t = 10$  seconds. The average waiting time (in seconds) for the 10 processes is \_\_\_\_\_. (rounded off to one decimal place)

[2026 (Set-1) : 2 M]

**1.77** Which one of the following CPU scheduling algorithms cannot be preemptive?

- Shortest Remaining Time First (SRTF) Scheduling
- First Come First Serve (FCFS) Scheduling
- Round Robin Scheduling
- Priority Scheduling

[2026 (Set-2) : 1 M]



## Answers | Process Management-I

|             |             |          |             |             |             |                |             |                |
|-------------|-------------|----------|-------------|-------------|-------------|----------------|-------------|----------------|
| 1.1 (b)     | 1.2 (a)     | 1.3 Sol. | 1.4 (a)     | 1.5 (c)     | 1.6 (c)     | 1.7 (c)        | 1.8 (d)     | 1.9 (b)        |
| 1.10 (b)    | 1.11 (a)    | 1.12 (d) | 1.13 Sol.   | 1.14 (d)    | 1.15 (a)    | 1.16 (a)       | 1.17 (c)    | 1.18 (a)       |
| 1.19 (d)    | 1.20 (c)    | 1.21 (d) | 1.22 (b)    | 1.23 (b)    | 1.24 (b)    | 1.25 (b)       | 1.26 (a)    | 1.27 (a)       |
| 1.28 (d)    | 1.29 (b)    | 1.30 (c) | 1.31 (b)    | 1.32 (b)    | 1.33 (c)    | 1.34 (d)       | 1.35 (c)    | 1.36 (d)       |
| 1.37 (c)    | 1.38 (a)    | 1.39 (c) | 1.40 (c)    | 1.41 (b)    | 1.42 (d)    | 1.43 (7.2)     | 1.44 (1000) | 1.45 (5.5)     |
| 1.46 (12)   | 1.47 (d)    | 1.48 (c) | 1.49 (a)    | 1.50 (8.25) | 1.51 (d)    | 1.52 (3)       | 1.53 (b)    | 1.54 (29)      |
| 1.55 (31)   | 1.56 (2)    | 1.57 (d) | 1.58 (5.25) | 1.59 (12)   | 1.60 (a, c) | 1.61 (b, c, d) | 1.62 (d)    | 1.63 (b, c, d) |
| 1.64 (c, d) | 1.65 (c, d) | 1.66 (c) | 1.67 (a, b) | 1.68 (d)    | 1.69 (b, d) | 1.70 (a, d)    | 1.71 (d)    | 1.72 (21)      |
| 1.73 (a)    | 1.74 (d)    | 1.75 (4) | 1.76 (9.5)  | 1.77 (b)    |             |                |             |                |

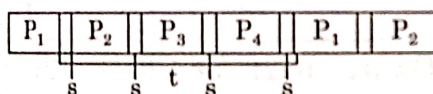
## Explanations | Process Management-I

1.1 (b)

Spooled devices is output device used to print the output of jobs.

1.2 (a)

Let,  $n = 4$



$$t \geq n \times s + q \times (n - 1)$$

So,  $q \leq \frac{t - ns}{n - 1}$

1.3 Sol.

The average response time depends on the value of  $x$ ,

If value of  $x < 3$

then order of execution should be

$x, 3, 5, 6$

If value of  $x > 3$  &  $x < 5$

then order of execution should be

$3, x, 5, 6$

If value of  $x > 5$  &  $x < 6$

then order of execution should be

$3, 5, x, 6$

If value of  $x > 6$

then order of execution should be

$3, 5, 6, x$

Note: Expected run times are in the increasing order.

1.4 (a)

System calls are invoked by using software interrupt.

1.5 (c)

The system has multi user and multi processing, the security is primary concern. Otherwise user program and operating system will be corrupted. That is the reason to ensure the security and protection hence two modes are required.

1.6 (c)

Swapping out the memory image of process A to the disk typically not performed by OS when switching context from process A to B.

1.7 (c)

To execute privileged instructions, system services can be obtained using software interrupt.

1.8 (d)

Because we want to change the mode from privileged to non-privileged, to the next instruction to be executed should be non-privileged instruction.

1.9 (b)

SJF executes shortest job very early so throughput increases in the case of SJF.

1.10 (b)

FCFS is non pre-emptive algorithm.

1.11 (a)

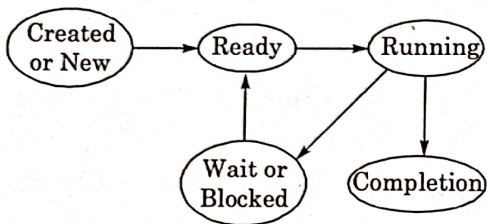
Device can not interrupt a running process but timer, scheduler and power failure can interrupt a running process.

## 1.12 (d)

- (i) Virtual memory helps to load more than one program.
- (ii) Multi programming, wait for an I/O of one process can execute other process.
- (iii) After completion of one process, other process executes, immediately.

**1.13 Sol.**

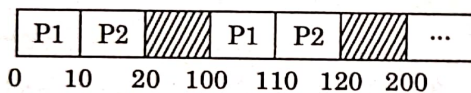
Just draw the diagram by using all criteria as below.



**1.14 (d)**

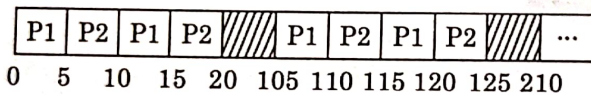
CPU = 10 ms  
I/O = 90 ms

**FCFS:**



So CPU idle time is 80

## Round Robin:



CPU idle time is 85.

Therefore, Round Robin scheduling with a time quantum of 5ms has least CPU utilization.

**1.15 (a)**

Consider the each statement:

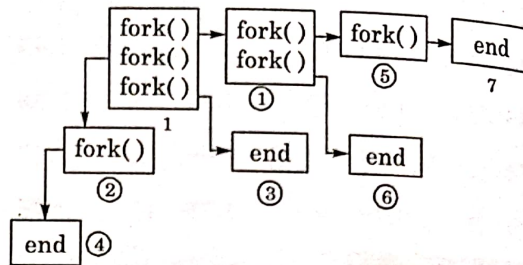
- (i) Statement is false because there is no connection between Kernel-supported threads and context switch.
- (ii) Statement is true because it is the drawback of user-level threads blocking system call can block the entire process.
- (iii) Statement is true because kernel-supported threads having own memory area and scheduled independently by the operating system.
- (vi) Statement is true because kernel is unaware about user level threads and there is no kernel support to user-level threads. So user-level threads are transparent to the Kernel.

## 1.16 (a)

Stack and registers are not shared by the threads of the same process while address space, message queue etc. are shared.

**1.17 (c)**

The number of process created will be  $2^n - 1$ .



$$2^3 - 1 = 7$$

## 1.18 (a)

| Process        | Arrival Time | Burst Time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0            | 5          |
| P <sub>2</sub> | 1            | 3          |
| P <sub>3</sub> | 2            | 3          |
| P <sub>4</sub> | 4            | 1          |

The Gantt chart for SRPT CPU scheduling algorithm is

|                |                |                |                |                |                |                |                |                |                |                |                |    |
|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----|
| P <sub>1</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>4</sub> | P <sub>3</sub> | P <sub>3</sub> | P <sub>3</sub> | P <sub>1</sub> | P <sub>1</sub> | P <sub>1</sub> | P <sub>1</sub> |    |
| 0              | 1              | 2              | 3              | 4              | 5              | 6              | 7              | 8              | 9              | 10             | 11             | 12 |

Process  $P_1$  arrive at time 0, at time unit 1 the remaining time of  $P_1$  is 4 and remaining time of  $P_2$  is 3 so  $P_1$  preempted. At the end of time unit 2 the remaining time of  $P_2$  is 2 and  $P_3$  is 3 so  $P_2$  is not preempted At time unit 4 the remain time of  $P_2$  is 0 and  $P_4$  is scheduled. The same process repeated upto time unit 12.

The turnaround time is the interval between submission and completion of a given process. Calculate turnaround time for each process as

$$P_1 = 12 - 0 = 12$$

$$P_2 = 4 - 1 = 3$$

$$P_3 = 6, P_4 = 1$$

So, Average Turn Around Time

$$= \frac{12 + 3 + 6 + 1}{4} = \frac{22}{4} = 5.50$$

**1.19 (d)**

Using preemptive priority scheduling.  
Gantt chart:

|    |    |    |    |    |
|----|----|----|----|----|
| P3 | P2 | P1 | P2 | P3 |
| 0  | 3  | 5  | 25 | 33 |
|    |    |    | 45 |    |

| Process | Priority | CPU Time | Arrival Time | TAT |
|---------|----------|----------|--------------|-----|
| P1      | 10       | 20 sec   | 00.00.05     | 20  |
| P2      | 9        | 10 sec   | 00.00.03     | 30  |
| P3      | 8        | 15 sec   | 00.00.00     | 45  |

So, Turn Around Time of P2 is 30.

Using non-preemptive priority scheduling.

Gantt chart:

|    |    |    |    |
|----|----|----|----|
| P3 | P1 | P2 |    |
| 0  | 15 | 35 | 45 |

| Process | Priority | CPU Time | Arrival Time | TAT |
|---------|----------|----------|--------------|-----|
| P1      | 10       | 20 sec   | 00.00.05     | 30  |
| P2      | 9        | 10 sec   | 00.00.03     | 42  |
| P3      | 8        | 15 sec   | 00.00.00     | 15  |

So, Turn Around Time of P2 is 42.

1.20 (c)

By following I and II conditions the process  $P_1$  will get both the resources  $R_1$  and  $R_2$ .

If  $R_1$  and  $R_2$  are allocated to the Process  $P_1$ , then it will complete its job and release it. After that process  $P_2$  will get both the resources and complete its job.

1.21 (d)

Fork() returns 0 is child process and process ID of child process is parent process.

In child(x),  $a = a + 5$

In parent(u),  $a = a - 5$

Therefore,  $x = u + 10$

The physical addresses of parent and child processes will be different,  $v \neq y$ .

1.22 (b)

Let three processes are  $P_1$ ,  $P_2$  and  $P_3$

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0            | 10         |
| $P_2$   | 2            | 20         |
| $P_3$   | 6            | 30         |

The Gantt chart for SRTF scheduling algorithm is

|       |       |       |    |
|-------|-------|-------|----|
| $P_1$ | $P_2$ | $P_3$ |    |
| 0     | 10    | 30    | 60 |

So there is only two context switches, at time unit 10 context switch from  $P_1$  to  $P_2$  and at time unit 30 context switch from  $P_2$  to  $P_3$ .

1.23 (b)

If it were a preemptive scheduling then there would have been a transition from Running state to Ready state. So it is non-preemptive scheduling.

1.24 (b)

At  $t = 0$  only  $P_1$  is present so execute it for 1 unit (remaining CPU time of  $P_1 = 0$ , now it will perform I/O).

At  $t = 1$  no process is available for CPU so CPU will remain idle and  $P_1$  will perform I/O (remaining I/O time of  $P_1 = 4$ ).

At  $t = 2$   $P_2$  arrives and is available for CPU so execute it for 1 unit and simultaneously  $P_1$  will do I/O (remaining CPU time of  $P_2 = 2$  & remaining I/O time of  $P_1 = 3$ ).

At  $t = 3$   $P_3$  also arrives and it is having highest priority among all processes available for CPU so we can execute  $P_3$  for its complete CPU burst (i.e 2 unit) as all processes has arrived and simultaneously  $P_1$  will do I/O (remaining CPU time of  $P_3 = 0$ , now  $P_3$  will perform I/O & remaining I/O time of  $P_1 = 1$ ).

At  $t = 5$  only  $P_2$  is available for CPU so it will execute for 1 unit and  $P_1$  and  $P_3$  will perform I/O (remaining CPU time of  $P_2 = 1$ , remaining I/O time of  $P_3 = 2$ , remaining I/O time of  $P_1 = 0$ , now  $P_1$  will perform CPU).

At  $t = 6$   $P_2$  and  $P_1$  are available for CPU but  $P_1$  will be selected because it is having highest priority then  $P_2$  so execute  $P_1$  for 1 unit (remaining CPU time of  $P_1 = 2$  & remaining I/O time of  $P_3$  is 1).

At  $t = 7$  again  $P_1$  will be executed for 1 unit (remaining CPU time of  $P_1 = 1$  & remaining I/O time of  $P_3 = 0$ , now  $P_3$  will perform CPU).

At  $t = 8$  now  $P_1$ ,  $P_2$  &  $P_3$  are available for CPU so  $P_3$  will be selected based on highest priority and will be executed for 1 unit (remaining CPU time of  $P_3 = 0$ , so  $P_3$  completed at  $t = 9$ ).

At  $t = 9$   $P_1$  will be executed on CPU for 1 unit (remaining CPU time of  $P_1 = 0$ , so  $P_1$  completed at  $t = 10$ ).

At  $t = 10$  only  $P_2$  is available for CPU so it will execute its remaining CPU burst (i.e 1 unit) (remaining CPU time of  $P_2 = 0$ , now  $P_2$  will perform I/O.)

At  $t = 11$   $P_2$  will perform I/O for 3 units and CPU will remain IDLE.

At  $t = 14$   $P_2$  will perform CPU again and  $P_2$  is completed at  $t = 15$

|       |      |       |       |       |       |       |       |       |       |      |       |
|-------|------|-------|-------|-------|-------|-------|-------|-------|-------|------|-------|
| $P_1$ | IDLE | $P_2$ | $P_3$ | $P_2$ | $P_1$ | $P_1$ | $P_3$ | $P_1$ | $P_2$ | IDLE | $P_2$ |
| 0     | 1    | 2     | 3     | 5     | 6     | 7     | 8     | 9     | 10    | 11   | 14 15 |

**1.25 (b)**

| Process | Total burst time | I/O time | CPU time | I/O time |
|---------|------------------|----------|----------|----------|
| P1      | 10               | 2        | 7        | 1        |
| P2      | 20               | 4        | 14       | 2        |
| P3      | 30               | 6        | 21       | 3        |

The Gantt chart is

|      |    |    |    |      |    |
|------|----|----|----|------|----|
| Idle | P1 | P2 | P3 | Idle |    |
| 0    | 2  | 9  | 23 | 44   | 47 |

Total time spent = 47

Idle time = 2 + 3 = 5

$$\% \text{ of idle time} = \frac{5}{47} \times 100 = 10.6\%$$

**1.26 (a)**

| Process id     | Arrival Time | CPU burst |
|----------------|--------------|-----------|
| P <sub>0</sub> | 0            | 2         |
| P <sub>1</sub> | 0            | 4         |
| P <sub>2</sub> | 0            | 8         |

The Gantt chart for LRTF CPU scheduling algorithm.

|                |                |                |                |                |                |                |                |                |                |                |                |                |                |                |
|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| P <sub>0</sub> | P <sub>1</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>2</sub> | P <sub>1</sub> | P <sub>2</sub> | P <sub>1</sub> | P <sub>2</sub> | P <sub>0</sub> | P <sub>1</sub> | P <sub>2</sub> |
| 0              | 1              | 2              | 3              | 4              | 5              | 6              | 7              | 8              | 9              | 10             | 11             | 12             | 13             | 14             |

Turn around time for P<sub>0</sub> = 12 - 0 = 12

Turn around time for P<sub>1</sub> = 13 - 0 = 13

Turn around time for P<sub>2</sub> = 14 - 0 = 14

$$\text{Average Turn around Time} = \frac{39}{3} = 13$$

**1.27 (a)**

List-I

- Gang Scheduling
- Rate Monotonic Scheduling
- Fair Show Scheduling

List-II

- Thread Scheduling
- Real-time Scheduling
- Guaranteed Scheduling

**1.28 (d)**

In Kernel level threads, the blocking system call causes, the Kernel can schedule another thread in the application for execution. So statement (d) is false about Kernel level threads.

**1.29 (b)**

| Process        | Execution Time | Arrival Time |
|----------------|----------------|--------------|
| P <sub>1</sub> | 20             | 0            |
| P <sub>2</sub> | 25             | 15           |
| P <sub>3</sub> | 10             | 30           |
| P <sub>4</sub> | 15             | 45           |

The Gantt chart for SRT Scheduling algorithm is

|                |                |                |                |                |    |
|----------------|----------------|----------------|----------------|----------------|----|
| P <sub>1</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>2</sub> | P <sub>4</sub> |    |
| 0              | 20             | 30             | 40             | 55             | 70 |

So the waiting time for

$$P_2 = (20 - 15) + (40 - 30) = 5 + 10 = 15$$

**1.30 (c)**

When time quantum used in round robin scheduling is more than maximum time required to execute any process then its behave like First Come First serve.

**1.31 (b)**

Statement (b) is not true because an ISR (Interrupt Service Routine) is invoked after completion of I/O in synchronous but not in asynchronous I/O.

**1.32 (b)**

fork( ) system call creates the child process initially number of processes is 0. After first fork(), it creates a single process. After second fork( ), it creates one parent and two child processes. After n + 1 fork(), the total number of processes is 2<sup>n</sup> but we subtract the main process then total number of child processes is 2<sup>n</sup> - 1.

**1.33 (c)**

Transition C indicates that OS uses preemptive scheduling. II is also true.

**1.34 (d)**

In the shortest remaining time first scheduling, process will not get a chance if that one has the high remaining time so due to this, starvation take place.

Shortest remaining time first scheduling will cause starvation, is true because if a large CPU burst required process is there and if consider that all the processes which are coming have less CPU-burst then the former have to wait until all finish execution gives starvation.

In preemptive scheduling, always one process preempt due to low priority then due to this starvation take place.

Response time = First response - Process submission time

In the round robin every process first response gets after certain time quantum so due to this it give more better than FCFS in term of response time.

1.35 (c)

Process switching includes mode switching.  
Context switching can occur only in Kernel mode.

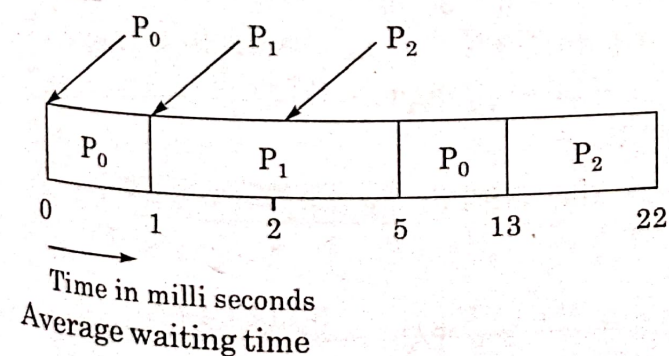
1.36 (d)

Interrupt from CPU temperature sensor is given top priority to protect system resources. When CPU temperature is too high, the BIOS initiate an interrupt and informs the Operating System. OS gives top priority to this interrupt and immediately shuts down the system.

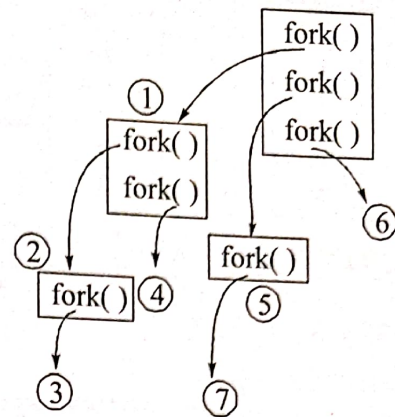
1.37 (c)

- (a) **False:** On per thread basics, OS maintains both CPU register state and stack. Hence option (a) is false, because only CPU register state mentioned.
- (b) **False:** OS maintains a separate stack for each thread.
- (c) **True:** OS does not maintain virtual memory state.
- (d) **False:** OS does not maintain scheduling and accounting information.

1.38 (a)



1.39 (c)



The number of child process created  
 $= 2^n - 1 = 2^3 - 1 = 7$   
 (where n is number of fork() statements)

1.40 (c)

RR Queue:

P1 P2 P1 P3 P2 P1 P3 P2  
 0 1 2 3 4 6 8 10

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| P1 | P2 | P1 | P3 | P2 | P1 | P3 | P2 |    |
| 0  | 2  | 4  | 6  | 8  | 10 | 11 | 13 | 16 |

$\Rightarrow$  P1, P3, P2

FCFS: P1, P2, P3

Therefore option (c) is correct.

FCFS : P1, P2, P3

RR2 : P1, P3, P2

1.41 (b)

Lets take an example:

| Process | Execution time | Arrival time |
|---------|----------------|--------------|
| 1       | 10             | 0            |
| 2       | 8              | 1            |
| 3       | 7              | 2            |
| 4       | 6              | 3            |

Consider scheduler schedule processes priority after S times units so order of execution will be  $P_1 P_2 P_3 P_4 P_1 P_2 P_3 P_4$  ( $S < \text{execution time of any process}$ ) which is exactly same as round robin so answer is (b).

1.42 (d)

Kernel level threads shares the code segment.

**1.43** (7.2)

|   | A.T | E.T |
|---|-----|-----|
| A | 0   | 6   |
| B | 3   | 2   |
| C | 5   | 4   |
| D | 7   | 6   |
| E | 10  | 3   |

Using SRTF:

|   |   |   |   |    |    |
|---|---|---|---|----|----|
| A | B | A | C | E  | D  |
| 0 | 3 | 5 | 8 | 12 | 15 |

$$\text{T.A.T (A)} = 8 - 0 = 8$$

$$\text{T.A.T (B)} = 5 - 3 = 2$$

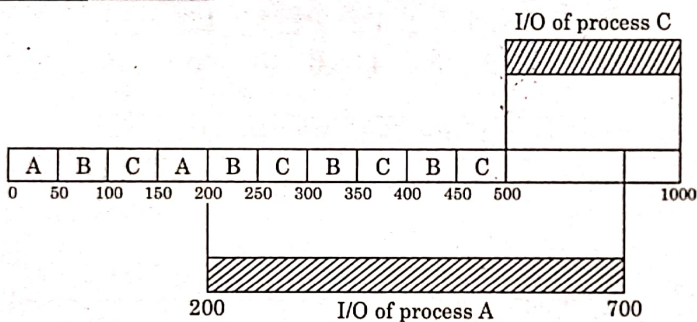
$$\text{T.A.T (C)} = 12 - 5 = 7$$

$$\text{T.A.T (D)} = 21 - 7 = 14$$

$$\text{T.A.T (E)} = 15 - 10 = 5$$

$$\text{Average T.A.T} = \frac{8+2+7+14+5}{5} = 7.2$$

**1.44 (1000)**



$\therefore$  At 1000 time units C completes its I/O.

**1.45 (5.5)**

**SRTF:**

|    |    |    |    |    |
|----|----|----|----|----|
| P1 | P2 | P3 | P4 | P1 |
| 0  | 2  | 6  | 12 | 17 |

Waiting time of P1 =  $17 - 2 = 15$

Waiting time of P2 =  $2 - 2 = 0$

Waiting time of P3 =  $6 - 3 = 3$

Waiting time of P4 =  $12 - 8 = 4$

$$\text{Average waiting time} = \frac{15 + 0 + 3 + 4}{4} = 5.5$$

1.46 (12)

Periodic arrival times of T1: 0, 3, 6, 9, 12, 15, 18, 21,...

Priority of  $T_1 = \frac{1}{3}$ , service time of  $T_1 = 1$ .

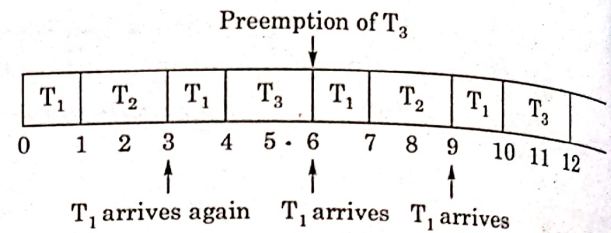
Periodic arrival times of  $T_2$ : 0, 7, 14, 21,...

Priority of  $T_2 = \frac{1}{7}$ , service time of  $T_2 = 2$ .

Periodic arrival times of  $T_3$ : 0, 20, 40,...

Priority of  $T_3 = \frac{1}{20}$ , service time of  $T_3 = 4$ .

$T_1$  has highest priority and  $T_3$  has lowest priority.



First instance of  $T_3$  (4 units) completed at the end of 12 ms.

**1.47 (d)**

Maximum number of processes that can be in ready state is independent of number of processes (n).

1.48 (c)

| Process  | Arrival Time | B.T |
|----------|--------------|-----|
| <i>A</i> | 0            | 3   |
| <i>B</i> | 1            | 6   |
| <i>C</i> | 4            | 4   |
| <i>D</i> | 6            | 2   |

(i) **FCFS:**

|   |   |   |   |
|---|---|---|---|
| A | B | C | D |
|---|---|---|---|

0      3      9      13      15

$$\text{Avg TAT} = \frac{3+8+9+9}{4} = \frac{29}{4} = 7.25$$

(ii) **Non-preemptive SJF:**

|   |   |   |    |
|---|---|---|----|
| A | B | D | C  |
| 0 | 3 | 9 | 11 |

$$\text{Avg TAT} = \frac{3+8+5+11}{4} = 6.75$$

(iii) **SRTF:**

|   |   |   |   |   |
|---|---|---|---|---|
| A | B | D | C | B |
|---|---|---|---|---|

  
0     3     4     8     10     15

$$\text{Avg TAT} = \frac{3+14+4+4}{4} = \frac{25}{4} = 6.25$$

(iv) **RR:**

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| A | B | A | C | B | D | C | B |
|---|---|---|---|---|---|---|---|

  
0    2    4    5    7    9    11    13    15

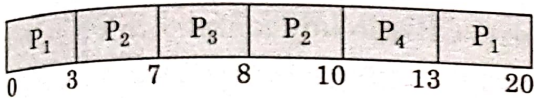
$$\text{Avg TAT} = \frac{5+14+9+5}{4} = \frac{33}{4} = 8.25$$

$\therefore$  SRTF has lowest turn around time.

1.49 (a) To minimize the average waiting time, we need to select the shortest remaining time process first, because all are arriving at the same time, and they have unequal CPU burst times. All other options will not minimize the waiting time. So, the answer is SRTF algorithm.

1.50 (8.25)

Gantt chart



| Process Number | Arrival time | Burst time | Completion time | TAT |
|----------------|--------------|------------|-----------------|-----|
| 1              | 0            | 10         | 20              | 20  |
| 2              | 3            | 6          | 10              | 7   |
| 3              | 7            | 1          | 8               | 1   |
| 4              | 8            | 3          | 13              | 5   |

Average turn around time :  $33/4 = 8.25$

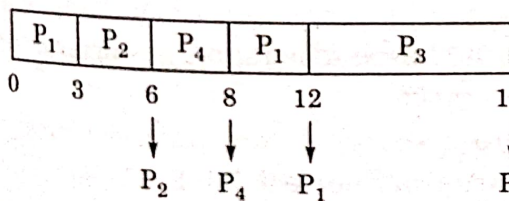
Average turn around time is 8.25.

1.51 (d)

Generally every thread of a process have their own PC and stack. Both heap and global variables are shared by every thread of a process.

1.52 (3)

Using pre-emptive SRTF algorithm Gantt chart will be,



Turnaround time

$$P_1 \rightarrow 12 - 0 = 12$$

$$P_2 \rightarrow 6 - 3 = 3$$

$$P_3 \rightarrow 17 - 5 = 12$$

$$P_4 \rightarrow 8 - 6 = 2$$

Waiting time

$$P_1 \rightarrow 12 - 7 = 5$$

$$P_2 \rightarrow 3 - 3 = 0$$

$$P_3 \rightarrow 12 - 5 = 7$$

$$P_4 \rightarrow 2 - 2 = 0$$

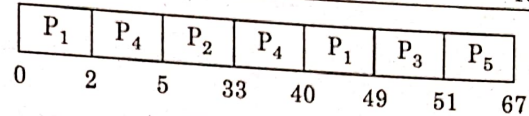
$$\text{Average Waiting Time} = \frac{5+0+7+0}{4} = 3.0$$

1.53 (b)

All the threads share address space but other entities like, stack, PC, registers are not shared and every thread will have its own.

1.54 (29)

| Process        | Arrival Time | Burst Time | Priority | C.T. | T.A.T. | W.T. |
|----------------|--------------|------------|----------|------|--------|------|
| P <sub>1</sub> | 0            | 11         | 2        | 49   | 49     | 38   |
| P <sub>2</sub> | 5            | 28         | 0 (high) | 33   | 28     | 0    |
| P <sub>3</sub> | 12           | 2          | 3        | 51   | 39     | 37   |
| P <sub>4</sub> | 2            | 10         | 1        | 40   | 38     | 28   |
| P <sub>5</sub> | 9            | 16         | 4*       | 67   | 58     | 42   |
| Total          |              |            |          |      |        | 145  |



$$\text{Average waiting time} = \frac{145}{5} = 29$$

1.55 (31)

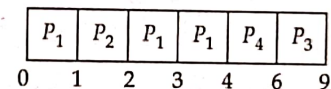
Number of child process created using  $n$  fork() calls =  $2^n - 1$

- fork(); statement inside the loop is executed when  $i = 0, 2, 4, 6, 8$ . So  $n = 5$  times fork() is executed.

So, number of child processes created =  $2^5 - 1 = 31$

1.56 (2)

Assume,  $Z = 2$



|                | Arrival time | CPU time | Completion time | Waiting time |
|----------------|--------------|----------|-----------------|--------------|
| P <sub>1</sub> | 0            | 3        | 4               | 1            |
| P <sub>2</sub> | 1            | 1        | 2               | 0            |
| P <sub>3</sub> | 3            | 3        | 9               | 3            |
| P <sub>4</sub> | 4            | Z = 2    | 6               | 0            |

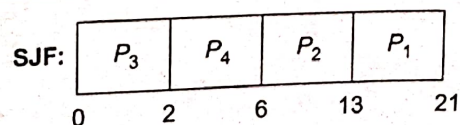
$$\text{Average waiting time (WT)} = \frac{1+0+3+0}{4} = 1 \text{ ms}$$

Hence,  $Z = 2$

1.57 (d)

Statement I, II and IV are correct.

1.58 (5.25)



$$\text{Turn Around Time (TAT)} = (21 - 0) + (13 - 0) + (2 - 0) + (6 - 0)$$

$$\text{Average TAT} = \frac{42}{4} = 10.5$$

RR:

|       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| $P_1$ | $P_2$ | $P_3$ | $P_4$ | $P_1$ | $P_2$ |
| 0     | 4     | 8     | 10    | 14    | 18    |

$$\begin{aligned}\text{Turn Around Time (TAT)} &= (18 - 0) + (21 - 0) \\ &+ (10 - 0) + (14 - 0) \\ &= 18 + 21 + 10 + 14\end{aligned}$$

$$\text{Average TAT} = \frac{63}{4} = 15.75$$

$$\begin{aligned}\text{Hence, } | \text{SJF (TAT)} - \text{RR(TAT)} | \\ = | 10.5 - 15.75 | = 5.25\end{aligned}$$

**1.59 (12)**

| WT | CT | P. No. | AT | BT |
|----|----|--------|----|----|
| 0  | 10 | A      | 0  | 16 |
| 26 | 46 | B      | 0  | 20 |
| 10 | 26 | C      | 0  | 10 |

|   |    |    |
|---|----|----|
| C | A  | B  |
| 0 | 10 | 26 |

$$\text{Average writing time} = \frac{0 + 26 + 10}{3} = \frac{36}{3} = 12$$

**1.61 (b, c, d)**

- Goal is to maximize CPU utilization and maximize the throughput. So, statement (a) is false.
- Statement (b) is true, because turnaround time = completion time – arrival time and waiting time is included in this.
- Statement (c) is true because using time quantum, we can run the processes even if burst time is not known initially in round-robin.
- True for example, round robin scheduling requires hardware support which is timer.

**1.62 (d)**

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| P | Q | R | S | Q | R | Q |
|---|---|---|---|---|---|---|

The above gantt chart satisfies all conditions given in the question the option (a), (b), (c) but not option (d). So, answer is option (d).

**1.63 (b, c, d)**

The status of a process or thread is saved via context switching. This makes it possible for the procedure to continue running later. During

context switches, the registers (integer and floating-point), programme counter, condition registers (status register or flag register), and any other thread execution state must be stored. Registers, the stack pointer, and the programme counter must all be reloaded from the TCB of the new thread.

When the threads are from different processes, it is necessary to load the pointer for the top-level page table of the new address space in addition to saving and restoring the information mentioned above. (Notice that the process control block (PCB) already contains this top-level page table pointer from the previous process, so it is not necessary to preserve it.)

**1.64 (c, d)**

Page fault and system call required to change mode but others may not required.

**1.65 (c, d)**

In First Come First Serve (FCFS), if a process with a very large burst time comes before other processes, the other processes will have to wait for a very long time, but it is obvious that other processes will definitely get their chance to execute, so it is not likely that it will suffer from starvation.

There is no starvation in Round Robin because there is a defined time duration and every process will have a chance to run.

If higher priority processes keep coming in priority-based scheduling, low priority processes will starve.

If processes with short process times continue to arrive in Shortest Job First (SJF), processes with longer burst times will have to wait and starve.

**1.72 (21)**

**Understanding the Process Execution:**

1. The program starts execution and enters the ready queue (this is not counted).
2. The first I/O operation occurs at `scanf("%d",&x);`, which takes input from the user.
  - The process moves to the I/O queue and gets context switched out.
  - Once the I/O is completed, the process returns to the ready queue.
3. The for loop runs 20 times, and in each iteration:

- The statement `printf("%d\n",x);` performs I/O operation.
- Each `printf()` results in an I/O request, causing the process to move to the I/O queue.
- After completing each I/O operation, the process returns to the ready queue.

### Counting the Number of Times the Process Enters the Ready Queue:

- 1<sup>st</sup> time: After `scanf()` completes  $\rightarrow$  1 time
- Each `printf()` (20 times) does I/O:
  - Each causes 1 exit to I/O queue and 1 re-entry to the ready queue.
  - Total for `printf()` statements: 20 times

Total Entries to Ready Queue (excluding the first time it starts running):

$$1 + 20 = 21$$

1.75 (4)

### Execution Trace:

#### 1. Iteration 1 ( $i = 0$ );

- Process  $P_0$  (Original) forks.
- Child  $C_1$  continues to iteration  $i = 1$ .
- Parent  $P_0$  breaks and prints "Hello!" (1).

#### 2. Iteration 2 ( $i = 1$ );

- Process  $C_1$  forks.
- Child  $C_2$  continues to iteration  $i = 2$ .
- Parent  $C_1$  breaks and prints "Hello!" (2).

#### 3. Iteration 3 ( $i = 2$ );

- Process  $C_2$  forks.
- Child  $C_3$  continues. Since the loop ends ( $i = 3$ ), it exits the loop naturally and prints "Hello!" (3).
- Parent  $C_2$  breaks and prints "Hello!" (4).

1.76 (9.5)

Burst time A = 6 sec, B = 8 sec

| Process | Arrival time | Burst time | CT | TAT | WT |
|---------|--------------|------------|----|-----|----|
| A       | 10           | 6          | 16 | 6   | 0  |
| C       | 11           | 8          | 24 | 13  | 5  |
| A       | 20           | 6          | 30 | 10  | 4  |
| C       | 22           | 8          | 38 | 16  | 8  |
| A       | 30           | 6          | 44 | 14  | 8  |
| C       | 33           | 8          | 52 | 19  | 11 |
| A       | 40           | 6          | 58 | 18  | 12 |
| C       | 44           | 8          | 66 | 22  | 14 |
| A       | 50           | 6          | 72 | 22  | 16 |
| C       | 55           | 8          | 80 | 25  | 17 |

CPU idle

|   |    |    |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|----|----|
|   | A  | C  | A  | C  | A  | C  | A  | C  | A  | C  |    |
| 0 | 10 | 16 | 24 | 30 | 38 | 44 | 52 | 58 | 66 | 72 | 80 |

Average waiting time

$$= \frac{0 + 5 + 4 + 8 + 8 + 11 + 12 + 14 + 16 + 17}{10} = \frac{95}{10} = 9.5$$

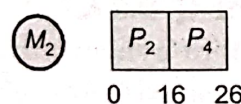
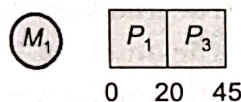
1.77 (b)

Non-preemptive Algorithm – First Come First Serve

Processes are executed in the order of Arrival time, once a process starts, it cannot be preempted until it finishes.



| P.No. | A.T. | B.T. | C.T. | T.A.T. | W.T. |
|-------|------|------|------|--------|------|
| $P_1$ | 0    | 20   | 20   | 20     | 0    |
| $P_2$ | 0    | 16   | 16   | 16     | 0    |
| $P_3$ | 0    | 25   | 45   | 45     | 20   |
| $P_4$ | 0    | 10   | 26   | 26     | 16   |

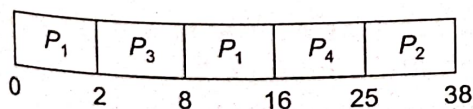


$$\text{Average waiting time} = \frac{0 + 0 + 20 + 16}{4} = \frac{36}{4} = 9$$

1.74 (d)

| Process | A.T. | B.T. | C.T. | T.A.T. |
|---------|------|------|------|--------|
| $P_1$   | 0    | 10   | 16   | 16     |
| $P_2$   | 1    | 13   | 38   | 37     |
| $P_3$   | 2    | 6    | 8    | 6      |
| $P_4$   | 8    | 9    | 25   | 17     |

Gantt Chart:



$$\text{Average Turn Around Time} = \frac{16 + 37 + 6 + 17}{4} = \frac{76}{4} = 19$$